

# Growtect Pulse — 統合システム要件定義書 v1.0

---

## コンテキスト（なぜ作るか）

---

日本の中小企業は「情シス担当者」に依存した属人的なIT運用を行っており、入退社・PC手配・アカウント管理・障害対応が担当者のスキルと記憶に紐づいている。Growtect Pulseはこの構造を破壊し、「管理」という人的負荷をAIとシステムに移管することで、人間が本来の業務に集中できる環境を実現する。

単なるチャットボットやSaaSではなく、**AI**を「脳」・既存**SaaS**群を「手足」として統制する「自律型コーポレート**OS**（**Autonomous Corporate OS**）」として設計する。

---

## 1. システムアーキテクチャ（4層構造）

---

|           |                                                 |
|-----------|-------------------------------------------------|
| L1 思考・判断層 | AIアドバイザー（ITIL 4ネイティブな意思決定）                      |
| L2 制御・指示層 | AIエグゼキューター（ワークフロー自動化・API連携）                     |
| L3 実装・実行層 | DRESS CODE / Intune / Google Endpoint（ポリシー自動適用） |
| L4 物理・実行層 | KDDIまとめてオフィス API（サイバーフィジカル同期）                   |

Pulseは「自社開発のHUB」として機能し、各層の外部システムとAPI通信を行う。

---

## 2. 機能要件

---

### 2-1. Pulse Desk（従業員向けヘルプデスク窓口）

インターフェース - Slack / Teams / Discord 上での自然言語による申請受付 - フロントエンドの対話・自動回答には **Zooba**（既存SaaS）を活用 - ZoobaはSlack/Teamsにネイティブ統合され、社内ナレッジ（Notion/Confluenceなど）を参照して一次回答する - Zoobaが解決できない依頼のみPulseオーケストレーターへエスカレーション

**AI**の主な責務 - 不足項目のヒアリング（追加情報の自動収集） - 設定・情報の確認（Read権限による台帳参照） - 申請アシスト（下書き生成・フォーム補完） - 進捗通知・完了連絡のSlackスレッド返信

チケット管理 - 正本：**Pulse**自前**DB** (PostgreSQL) に「最小で成立する依頼レコード」を保持 - LMIS（ユニリタ）はインシデント管理・CMDB・承認ワークフローの詳細管理に連携（参照/書き込みはAPI経由） - ステータス遷移：

受付 → 分析中 → 承認待ち → 承認済み → 実行中 → 完了 / 却下 / 否認 / エスカレーション - チケットに紐づく情報：会話サマリー・AI判断根拠・監査ログ・承認者・実行パラメータ

## 2-2. Agent Console（運用者向け管理画面）

- ・キュー管理（承認待ちタスクの一覧・優先度表示）
- ・タイムライン表示（チケットの状態遷移履歴）
- ・AIが作成した下書き・提案の確認・編集
- ・引き継ぎ機能（担当者変更）
- ・監査ログの閲覧・エクスポート

## 2-3. AIオーケストレーション（多段エージェント構成）

オーケストレーター（入口の司令塔） - Zooba/Slackからのイベントを受信し、ITIL 4規律に基づいて依頼を分類・ルーティング - Read操作は自律実行。Write操作は `ActionProposal` として生成し、承認ゲートへ送付 - ITIL規律違反・セキュリティリスク依頼は `reject_request` で即座に却下（監査役AI）

技術エージェント群 | エージェント | 責務 | |---| | InfraOps | ネットワーク障害・端末不調・接続問題の切り分け診断 | | SystemOps | SaaSアプリケーションの設定確認・運用判断 | | IAM Agent | Google Workspace / M365 アカウント作成・削除・権限変更 | | Google Workspace Agent | Google Admin SDK 経由の詳細操作 | | M365 Agent | Microsoft Graph API 経由の詳細操作 |

プロセス管理エージェント群 | エージェント | 責務 | |---| | Inc/Prob | トラブル止血（一時対応） → RCA → 恒久対策 → ナレッジ化 | | Change/Release | 変更影響範囲調査・手順作成・切り戻し・事後チェック管理 | | Project推進 | 新SaaS導入・全社移行の要件整理・タスク分解・進捗管理 |

バックオフィス拡張エージェント群 | エージェント | 責務 | |---| | AP/Billing Ops | メール/フォルダから請求書受領 → OCR抽出（金額・ベンダー・期限） → CMDB照合 → 担当者自動振り分け | | Procurement/VendorOps | KDDI等への発注要件定義・社内承認・発注構造化メール生成・進捗追跡・納品後台帳更新 | | FinOps-lite | IT支出ダッシュボード・ベンダー/契約/実績/配賦/予算管理・未使用ライセンス検知 |

## 2-4. 台帳レイヤ (Single Source of Truth)

全エージェントの判断根拠として機能する3つの台帳：

| 台帳               | 管理対象                           |
|------------------|--------------------------------|
| CMDB (構成管理)      | サービス・システム・デバイス・依存関係・オーナー・契約更新日 |
| IAM (権限・アカウント管理) | 従業員・ロール・付与ライセンス数・例外承認ルール       |
| Version (リリース台帳) | 環境別バージョン・リリース履歴・切り戻しポイント       |

## 2-5. 外部システム連携一覧

| カテゴリ      | システム             | 連携方式                             |
|-----------|------------------|----------------------------------|
| 対話フロント    | Zooba            | Webhook → Pulse API              |
| ITSM/台帳   | LMIS (ユニリタ)      | REST API (Salesforce基盤)          |
| 調達・物流     | KDDIまとめてオフィス     | REST API (発注・追跡)                 |
| エンドポイント管理 | DRESS CODE       | API (ゼロタッチデプロイ・プロファイル適用)         |
| エンドポイント管理 | Microsoft Intune | Microsoft Graph API              |
| エンドポイント管理 | Google Endpoint  | Google Admin SDK                 |
| IDaaS     | Google Workspace | Google Admin SDK / Directory API |
| IDaaS     | Microsoft 365    | Microsoft Graph API              |

**Read/Write分離の原則：**設定確認はRead権限で自律実行。Write (変更・作成・削除) は承認ゲート通過後のみ実行。

## 3. 非機能要件

### 3-1. AI推論エンジン (マルチLLM構成)

| ロール     | プロバイダー                 | 用途                           |
|---------|------------------------|------------------------------|
| Primary | Azure OpenAI (GPT-4o系) | 主推論エンジン。国内リージョン。エンタープライズSLA。 |

| ロール       | プロバイダー                               | 用途                                 |
|-----------|--------------------------------------|------------------------------------|
| Secondary | AWS Bedrock (Claude claude-opus-4-6) | フォールバック。AzureダウンまたはClaude特性が必要な処理。 |

- プロバイダー障害時は自動フェイルオーバー ( `tenacity` リトライ + 切り替えロジック )
- モデルインターフェースを抽象化し、呼び出し側がプロバイダーを意識しない設計
- 入力データは**AI**の学習に使用されないことを技術的・契約的に保証 (Azure / Bedrock のエンタープライズ契約)

### 3-2. データ保護

- **PII**マスキング：個人情報 (氏名・メールアドレス・電話番号) はAIへ渡す前にマスキング処理
- データ国内保持：全データを国内リージョン (Azure Japan East / AWS ap-northeast-1) に限定
- **API**トークン暗号化：SaaSのAPIトークンは `cryptography.Fernet` (AES-128-CBC) で暗号化保存。本番環境ではAzure Key Vault / AWS KMS (HSM相当) で鍵管理

### 3-3. 認証・アクセス制御

- **MFA + SSO**：全アクセスにMFAを必須。SAML/OIDC連携でシングルサインオン
- **Just-In-Time (JIT)** アクセス制御：AIがAPI実行する瞬間だけ必要なスコープを動的発行し、作業後に剥奪 (ゼロトラスト)
- 承認権限階層： `none < it_manager < department_head < ciso`

### 3-4. 可用性・整合性

- マルチ**LLM**フォールバック：Primary/Secondaryの自動切り替えで業務継続
- ドリフト検知：DRESS CODE / Intune から1時間ごとに自動ポーリングし、台帳と物理デバイスの差分を検知して修正提案を生成
- **Celery**非同期処理：Slackの3秒タイムアウト制約をCeleryタスクキューで回避

## 4. 運用・コンプライアンス要件

### 4-1. Human-in-the-Loop (HITL) の徹底

- AIによるWrite操作（API実行・設定変更・アカウント操作）は必ず `PENDING_APPROVAL` を経由
- Slackのインタラクティブメッセージ（Block Kit）で承認/却下ボタンを提示
- 承認者は権限レベルを事前登録。権限不足者の承認は拒否
- 承認期限：24時間。期限切れはエスカレーション

### 4-2. Policy-as-Code

- 業務ルール・法制度（インボイス制度・電帳法等）をYAMLコードとして管理
- 法改正時は基盤コードを修正するだけで全エージェントの判断基準が即時更新

### 4-3. 監査ログの完全性

- 「誰が・いつ・何を見て・何を承認・実行したか」を `audit_logs` テーブルに全記録
- AI分析ターン単位でモデル名・入出力トークン数・推論サマリーを保存
- ログの改ざん不可設計（追記のみ）

### 4-4. コスト提示 (Cost-Aware Logic)

- AIが提案する操作には、それによるSaaSコスト・API利用料のランニングコスト増減を必ず併記

## 5. テクノロジースタック

| 層                       | 技術                                    |
|-------------------------|---------------------------------------|
| AI推論 (Primary)          | Azure OpenAI (GPT-4o-mini / GPT-4o)   |
| AI推論 (Secondary)        | AWS Bedrock (Claude claude-opus-4-6)  |
| バックエンド                  | Python 3.12 + FastAPI + Celery        |
| DB                      | PostgreSQL 15 (Supabase互換) + pgvector |
| メッセージキュー                | Redis                                 |
| Slack連携                 | Slack Bolt for Python                 |
| フロントエンド (Agent Console) | Next.js 15 + shadcn/ui                |

| 層        | 技術                                                |
|----------|---------------------------------------------------|
| インフラ（開発） | Docker Compose                                    |
| インフラ（本番） | Azure Container Apps / AWS ECS                    |
| 暗号化      | cryptography (Fernet) + Azure Key Vault / AWS KMS |
| ログ       | structlog (JSON形式)                                |
| マイグレーション | Alembic                                           |

## 6. DBスキーマ（主要テーブル）

```

tickets                チケット（依頼レコード）正本
  id, subject, body, status, priority
  slack_channel_id, slack_thread_ts
  requester_email, requester_user_id
  ai_analysis (JSONB)

approvals              Human-in-the-loop承認レコード
  id, ticket_id, approver_user_id
  action_proposal (JSONB) ← AIが生成したWrite操作提案
  status, approver_comment
  slack_message_ts, expires_at

audit_logs             全操作の監査ログ（追記のみ）
  id, ticket_id, actor_type (ai/human/system), actor_id
  event_type, event_data (JSONB)
  ai_reasoning, ai_model_version

cmdb_devices           デバイス台帳
  id, hostname, serial_number, asset_tag
  device_type, os_type, os_version
  status, assigned_user_id
  intune_device_id, metadata (JSONB)

cmdb_users             ユーザーアカウント台帳
  id, email, full_name, employee_id
  department, slack_user_id, google_user_id
  approval_authority (none/it_manager/department_head/ciso)

cmdb_services          SaaSサービス台帳
  id, name, vendor, license_count, license_used
  api_credentials_encrypted, api_base_url
  contract_url, service_config (JSONB)

```

cmdb\_user\_service\_accounts    ユーザー×サービス紐付け

id, user\_id, service\_id, role\_in\_service, is\_active

## 7. 実装フェーズ計画

### Phase 1 — Pulse Desk MVP（ヘルプデスク中核）

目標：Slackから依頼を受け、AIが分析・提案し、人間が承認してサブエージェントが実行する最小ループ

| ステップ   | 実装内容                                           | 確認方法                                  |
|--------|------------------------------------------------|---------------------------------------|
| Step 1 | Docker環境・FastAPI起動・PostgreSQL・Redis            | GET /health が200                      |
| Step 2 | CMDB台帳スキーマ + Alembic + REST API                | デバイス登録・取得API動作                        |
| Step 3 | Slack Bolt受付 + チケット作成                          | @PulseBot メンション → DBにチケット生成           |
| Step 4 | Orchestratorエージェント + Celery 非同期                | AI分析 → 監査ログ記録                         |
| Step 5 | 承認フロー（Block Kit） + 権限チェック                      | 承認/却下ボタン → 状態遷移                       |
| Step 6 | IAM / InfraOps サブエージェント + Google Workspace SDK | 「アカウント作成」依頼 → 承認 → Google Admin API実行 |

### Phase 2 — バックオフィス自動化

- AP/Billing Ops：メール受信 → OCR（Claude Vision） → CMDB照合 → 振り分け
- Procurement/VendorOps：発注フロー → KDDI API連携
- LMIS連携：詳細インシデント管理・CMDB同期

### Phase 3 — ガバナンス・フルスタック

- Agent Console（Next.js）：承認キュー・タイムライン・監査ログUI
- Policy-as-Code（YAML規則エンジン）
- FinOps-lite：IT支出ダッシュボード・ライセンス最適化
- DRESS CODE連携：ドリフト検知・ゼロタッチデプロイ
- マルチLLMフォールバック完全実装

## 8. 重要設計決定 (ADR)

| 決定事項    | 選択                       | 理由                           |
|---------|--------------------------|------------------------------|
| チケット正本  | Pulse自前DB                | LMIS依存リスク回避。Pulseがデータ主権を持つ   |
| AI推論    | マルチLLM (Azure + Bedrock) | 単一障害点排除・コンプライアンス (データ非学習保証)  |
| 対話フロント  | Zooba連携                  | Zoobaが一次回答をさばき、Pulseは複雑処理に特化 |
| Write実行 | HITL必須 (承認ゲート)           | ハルシネーションによる誤操作・誤発注を防止        |
| 非同期処理   | Celery + Redis           | Slackの3秒タイムアウト制約の回避          |
| 暗号化     | Fernet + Key Vault/KMS   | SaaS APIトークンの平文保存を禁止         |

## 9. 検証方法 (End-to-End テスト)

1. **Slack**メンション受付 → チケットが `RECEIVED` でDB保存される
2. **Celery**タスク実行 → `ANALYZING` に遷移 → Azure OpenAI APIが呼ばれる → 監査ログに記録
3. **ActionProposal**生成 → `PENDING_APPROVAL` に遷移 → SlackにBlock Kitメッセージが届く
4. 承認ボタン押下 → 権限チェックOK → `APPROVED` → 実行タスクキューイング
5. 実行完了 → `COMPLETED` → Slackスレッドに完了通知
6. 却下ボタン → 否認理由モーダル → `DENIED` → チケットクローズ
7. **ITIL**違反依頼 → `reject_request` → Slackに却下理由を返信

## 11. ドキュメント出力手順

### MDファイル

```
/home/user/Factory/docs/growtect-pulse-requirements-v1.md
```

上記パスにコピー (Factory配下で版管理)。



## PDFファイル

pandocが未インストールのため、Python (markdown + weasyprint) で生成：

```
/home/user/Factory/docs/growtect-pulse-requirements-v1.pdf
```

---

## 10. 未確定・今後詰める事項

---

- Zooba → Pulse間のWebhook仕様詳細 (ZoobaのAPIドキュメント要確認)
- LMIS REST APIのエンドポイント仕様 (ユニリタに確認)
- DRESS CODEのAPI提供有無・仕様 (ベンダー確認)
- KDDIまとめてオフィスのAPI仕様・契約要件 (KDDI担当者確認)
- 本番インフラ：Azure Container Apps vs AWS ECS 最終選定